


SAGAB

Manual técnico

Arquitectura, instalación, configuración, datos, API, seguridad, operación, pruebas, despliegue y límites técnicos.

Producto: Sistema Avanzado de Gestión Académica Bellini

Versión documental: 3.1

Línea base inspeccionada: commit `7d21f8c` y árbol local al 7 de agosto de 2026

Audiencia: Desarrollo, soporte, infraestructura, seguridad y auditoría técnica

Documento técnico de uso controlado. No debe contener contraseñas, secretos, tokens, comprobantes ni datos personales reales.

Control y alcance

Elemento	Definición
Objetivo	Permitir que un equipo técnico instale, entienda, mantenga, asegure, pruebe y opere SAGAB sin depender de conocimiento oral.
Incluye	Frontend, backend, PostgreSQL, archivos, correo opcional, API, perfiles, procesos programados, controles, despliegue y recuperación.
No incluye	Credenciales reales, contratos de servicio, inventario del proveedor, políticas legales propias del colegio ni una certificación de seguridad.
Regla de actualización	Actualizar después de modificar endpoints, entidades, migraciones, permisos, variables, infraestructura o reglas de negocio.

Estado de madurez. La solución es una base funcional amplia, pero no debe calificarse como lista para producción institucional hasta cerrar los hallazgos críticos del capítulo 18 y ejecutar pruebas integrales de autorización, respaldo, carga y recuperación.

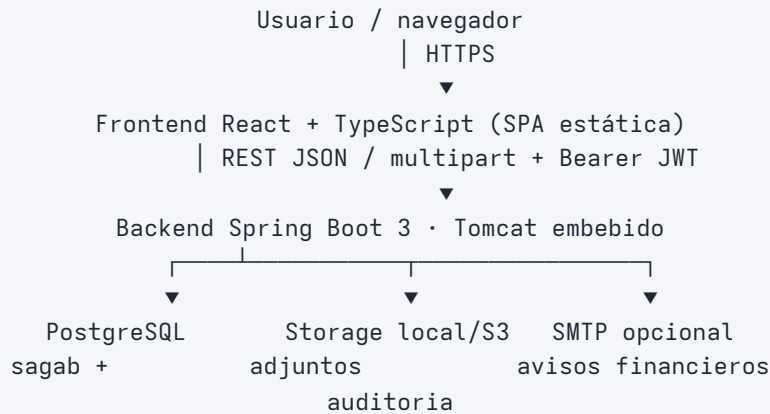
Contenido

1. Visión general y arquitectura
2. Tecnologías y dependencias
3. Estructura del repositorio
4. Arquitectura del backend
5. Arquitectura del frontend
6. Base de datos y modelo de información
7. Seguridad y autorización
8. Reglas de negocio
9. Catálogo de API
10. Almacenamiento y archivos
11. Configuración y variables
12. Instalación local
13. Despliegue de producción
14. Operación, respaldo y recuperación
15. Pruebas y calidad
16. Diagnóstico y errores

- 17. Capacidades y límites
- 18. Riesgos, deuda técnica y hoja de ruta
- 19. Listas de verificación

1. Visión general y arquitectura

SAGAB es una aplicación web de tres capas. El navegador ejecuta una SPA React; esta consume una API REST Spring Boot mediante JSON o `multipart/form-data`; la API persiste información en PostgreSQL y archivos en un directorio persistente o proveedor compatible con S3.



1.1 Responsabilidad de cada capa

Capa	Responsabilidades	No debe asumir
Frontend	Presentación, navegación, captura, validación inmediata, estado de sesión, descargas y experiencia por rol.	No es una frontera de seguridad. Ocultar un botón no autoriza ni protege un dato.
Backend	Autenticación, autorización, validación definitiva, reglas de negocio, archivos, reportes, auditoría y transacciones.	No debe confiar en IDs, roles, tipos MIME o valores calculados enviados por el cliente.
PostgreSQL	Integridad referencial, restricciones, índices, auditoría, estados y persistencia transaccional.	No reemplaza las comprobaciones de propiedad contextual realizadas en servicios.
Infraestructura	HTTPS, DNS, disponibilidad, secretos, red, copias, almacenamiento, métricas y alertas.	No forma parte automáticamente del código de aplicación.

1.2 Flujo normal de una petición

1. El usuario inicia sesión mediante `POST /api/auth/login`.
2. El servidor verifica usuario, estado, bloqueo y hash BCrypt; devuelve JWT y datos mínimos de sesión.
3. El frontend guarda el JWT en `sessionStorage` y adjunta `Authorization: Bearer ...`.
4. El filtro JWT valida firma/expiración y construye la identidad de Spring Security.
5. Las reglas HTTP, `@PreAuthorize` y el servicio validan rol y alcance.

6. Repositorio/JPA ejecuta la transacción; restricciones y triggers completan la integridad/auditoría.
7. La API devuelve DTO JSON, descarga binaria o error normalizado sin traza interna.

2. Tecnologías y dependencias

2.1 Backend

Componente	Versión/línea	Uso
Java	17	Lenguaje y runtime LTS.
Spring Boot	3.3.5	Aplicación, servidor embebido y configuración.
Spring Web	Gestionada por Boot	Controladores REST, JSON y multipart.
Spring Data JPA	Gestionada por Boot	Persistencia y repositorios.
Spring Security	Gestionada por Boot	Filtros, roles y seguridad de métodos.
JWT	0.12.6	Emisión y validación de tokens.
OpenPDF	1.3.30	Generación de papeletas/reportes PDF.
AWS SDK S3	2.51.0	Almacenamiento S3 compatible.
Lombok	1.18.38	Reducción de código repetitivo.
Maven	3.9 recomendado	Dependencias, pruebas y empaquetado.

2.2 Frontend

Componente	Versión	Uso
React / React DOM	18.3.1	Componentes y renderizado.
TypeScript	5.7	Tipado estático.
Vite	6.4.3	Desarrollo y compilación.
React Router	6.30.x	Rutas de la SPA.
Tailwind CSS	4.1.12	Estilos utilitarios.
CSS de formularios	Subconjunto local	Componentes necesarios de Matrícula y Portal Familiar, aislados bajo <code>.matricula-form</code> ; se retiró del bundle el Bootstrap completo.
Recharts	2.15.2	Gráficos del panel.
jsPDF / AutoTable	4.2.1 / 5.0.8	Reportes generados en el cliente.

Componente	Versión	Uso
ExcelJS	4.4.0	Exportación tabular donde se utilice.

2.3 Persistencia

PostgreSQL 15 o superior es la recomendación operativa. Se utilizan dos esquemas: `sagab` para negocio y `auditoria` para trazabilidad. Hibernate trabaja con `ddl-auto: validate`; no crea ni modifica el esquema.

3. Estructura del repositorio

```
sagab/
├── backend/
│   ├── Dockerfile
│   ├── pom.xml
│   └── src/
│       ├── main/java/ec/edu/bellini/sagab/
│       │   ├── config/      seguridad, storage, schedulers
│       │   ├── controller/  endpoints REST
│       │   ├── dto/         contratos de entrada/salida
│       │   ├── exception/   traducción de errores
│       │   ├── middleware/  filtro JWT
│       │   ├── model/       entidades JPA
│       │   ├── repository/  acceso a datos
│       │   ├── service/     reglas y autorización contextual
│       │   └── utils/       utilidades
│       └── test/            pruebas unitarias actuales
├── frontend/
│   ├── package.json
│   ├── vite.config.ts
│   ├── scripts/
│   └── src/
│       ├── api/             cliente HTTP por dominio
│       ├── app/components/  navegación y componentes
│       ├── app/hooks/       lógica reutilizable
│       ├── app/views/       pantallas por módulo
│       └── styles/           CSS global/aislado
├── database/
│   ├── setup_db.sh
│   └── 01_schema.sql ... 29_seguimiento_dece.sql
├── docs/                    manuales e informes
├── render.yaml              plantilla parcial de despliegue
└── README.md
```

3.1 Convenciones

- El código backend usa el paquete raíz `ec.edu.bellini.sagab`.
- Los endpoints de aplicación empiezan con `/api`.
- La interfaz utiliza importación diferida de vistas para separar chunks.
- Los cambios estructurales de datos deben agregarse como scripts idempotentes y probarse en una base limpia.
- Los datos demo o personales nunca deben ejecutarse automáticamente en producción.

4. Arquitectura del backend

4.1 Capas

Capa	Función	Regla de mantenimiento
Controller	Mapea HTTP, valida DTO, obtiene identidad y define rol base.	No colocar consultas complejas ni decisiones de propiedad.
DTO	Define contratos y validaciones de entrada/salida.	No exponer entidades JPA directamente.
Service	Aplica transacciones, reglas, ownership, archivos, mensajes y reportes.	Es la autoridad de negocio; validar toda referencia recibida.
Repository	Consultas JPA o nativas optimizadas.	Incluir filtros de activo/alcance y evitar N+1.
Model	Mapea tablas, relaciones, enumeraciones y columnas.	Mantener sincronía exacta con scripts y <code>ddl-auto: validate</code> .
Config/middleware	Seguridad, CORS, storage, secretos y tareas programadas.	Los valores operativos deben provenir del entorno.

4.2 Manejo de errores

Los errores de negocio y validación se convierten a respuestas HTTP mediante el paquete `exception`. El servidor tiene `server.error.include-stacktrace=never`, por lo que una respuesta no debe revelar SQL, rutas, clases ni secretos. El frontend busca el campo `mensaje`; si no existe, presenta el estado HTTP.

Estado	Interpretación operativa
400	Solicitud inválida, dato fuera de rango o regla de negocio incumplida.
401	Token ausente, inválido o expirado. El cliente limpia la sesión activa.
403	Rol o propiedad insuficiente.
404	Recurso inexistente o no visible para esa identidad.
409	Conflicto o duplicado lógico.
413	Archivo o petición demasiado grande, según infraestructura/configuración.
500	Error no controlado; correlacionar hora y usuario con logs sin entregar detalles internos al cliente.

4.3 Procesos programados

Proceso	Programación	Acción
Obligaciones vencidas	Al arrancar y cada día a las 02:00.	Cambia obligaciones pendientes cuya fecha venció a <code>VENCIDO</code> .
Partición de auditoría	Al arrancar y el día 1 de cada mes a las 03:00.	Crea preventivamente la partición mensual necesaria.

La zona horaria JDBC está configurada como `America/Guayaquil`. Debe alinearse con la zona del contenedor y de PostgreSQL para evitar cortes de fecha inconsistentes.

5. Arquitectura del frontend

5.1 Arranque y sesión

`App.tsx` recupera la sesión con `/api/auth/me`, fuerza la pantalla de cambio de clave cuando corresponde y elige la ruta inicial según el primer rol recibido. El token se guarda en `sessionStorage` bajo `sagab_token`; por tanto no se comparte entre sesiones de navegador y se elimina al cerrar sesión o recibir 401.

Decisión actual. No se utiliza refresh token en el flujo activo, aunque existe una tabla de base para ello. Al vencer el JWT de 30 minutos, la persona debe autenticarse otra vez.

5.2 Rutas y perfiles

Ruta	Vista	Roles de navegación
<code>/dashboard</code>	Panel	ADMIN, DOCENTE, AUDITOR
<code>/matricula</code>	Matrícula	ADMIN
<code>/grades</code>	Académico	ADMIN, DOCENTE
<code>/attendance</code>	Asistencia	ADMIN, DOCENTE
<code>/calendar</code>	Calendario	ADMIN, DOCENTE, DECE
<code>/messages</code>	Mensajes institucionales	ADMIN
<code>/tareas</code>	Deberes	DOCENTE
<code>/financiamiento</code>	Financiero	ADMIN
<code>/personal</code>	Personal	ADMIN
<code>/deceSeguimiento</code>	Seguimientos	DECE
<code>/deceAlertas</code>	Alertas de asistencia	DECE
<code>/auditoria</code>	Auditoría	AUDITOR
<code>/parent</code>	Portal familiar integrado	REPRESENTANTE, ESTUDIANTE

Las rutas usan guardas visuales y redirección, pero la autorización real debe mantenerse en el backend. Representante y Estudiante utilizan un layout propio; no usan el Sidebar institucional.

5.3 Cliente HTTP

- `api()`: JSON, agrega Bearer y normaliza errores.
- `apiForm()`: `FormData`; no fija manualmente Content-Type para conservar el boundary.
- `apiBlob()`: reportes/descargas autenticadas.
- Un 401 con token previo dispara `sagab:sesion-expirada`.
- La URL base proviene de `VITE_API_URL`, con fallback `http://localhost:8080`.

5.4 Compilación

Las vistas son chunks diferidos. Vite desactiva mapas de fuente de producción y separa algunas dependencias pesadas. Se sustituyó el CSS Bootstrap completo de aproximadamente 280 KB por `forms-scoped.css` y las fuentes Inter se limitaron a Latin; el directorio de producción bajó aproximadamente de 3,2 MB a 2,2 MB. El chunk de gráficos sigue cerca de 553 KB antes de gzip, pero solo se solicita al abrir el panel.

Medición Lighthouse. No evaluar `npm run dev` con extensiones del navegador activas. La captura de referencia con puntuación 45 indicaba interferencia de extensiones. La medición válida debe hacerse contra el build de producción, en incógnito o en un perfil limpio, y repetirse varias veces.

6. Base de datos y modelo de información

6.1 Esquemas y grupos de tablas

Dominio	Tablas principales	Relación central
Identidad	rol, permiso, rol_permiso, usuario, usuario_rol, refresh_token	Usuario recibe uno o más roles; refresh_token no se usa en el flujo actual.
Estructura académica	periodo_academico, paralelo, materia, docente, asignacion_docente	Asignación = docente + materia + paralelo + periodo.
Familia	representante, estudiante	Estudiante se vincula a usuario, representante y paralelo.
Académico	calificacion, asistencia, incidencia_disciplinaria	Notas y asistencia enlazan estudiante con asignación/paralelo.
Deberes y recursos	tarea, tarea_adjunto, entrega_tarea, recurso_academico	Docente publica por asignación; estudiante entrega por tarea.
Finanzas	rubro, obligacion_pago, pago	Obligación pertenece a estudiante/rubro; pagos aplican a obligación.
Comunicación	mensaje, mensaje_destinatario, notificacion	Separación de contenido, destinatarios y lectura.
Calendario	evento_calendario, evento_calendario_adjunto	Evento institucional con estado, rango y adjuntos.
DECE	seguimiento_dece, seguimiento_dece_historial, seguimiento_dece_mensaje	Caso por estudiante con línea de tiempo y comunicación.
Auditoría	auditoria.registro_cambio, auditoria.evento_seguridad	Cambios before/after y sucesos de seguridad.

6.2 Relaciones y restricciones importantes

```

representante 1 — * estudiante * — 1 paralelo
                    |
                    |— * calificacion * — 1 asignacion_docente
                        |
                        |— * asistencia
                            |
                            |— * obligacion_pago 1 — * pago
                                |
                                |— * entrega_tarea * — 1 tarea
                                    |
                                    |— * seguimiento_dece

docente 1 — * asignacion_docente * — 1 materia
                    |
                    |— 1 paralelo
                        |
                        |— * tarea
                            |
                            |— * recurso_academico

```

- Asignación única por docente/materia/paralelo/periodo, según el esquema.
- Calificación única por estudiante/asignación/parcial.
- Índices parciales aceleran usuarios no activos, estudiantes activos, notas en riesgo, ausencias injustificadas, mora y pagos en revisión.
- Un trigger valida que la suma de pagos aprobados no supere la obligación.
- Las tablas auditadas usan triggers de cambio; las tablas de auditoría bloquean UPDATE y DELETE.
- `registro_cambio` está particionada por mes e incluye una partición default.

6.3 Instalación del esquema

`database/setup_db.sh` crea la base local si hace falta y aplica **23 scripts estructurales**. El orden no es el orden numérico literal: `09_estudiante_usuario.sql` debe ejecutarse antes de `07_usuario_username.sql`; el script ya contiene el orden verificado.

No ejecutar datos demo en producción. Los scripts 05, 08, 11 y 24 son datos/cuentas de prueba. Los scripts 07 y 10 contienen cuentas personales y deben revisarse, sanearse o sustituirse por un aprovisionamiento controlado.

El proyecto no utiliza Flyway ni Liquibase. La versión instalada debe registrarse externamente y cada cambio debe probarse contra una copia y una base vacía.

6.4 Roles de base de datos

Rol	Finalidad	Recomendación
<code>sagab_app</code>	Conexión normal de aplicación; CRUD en negocio.	Rotar contraseña semilla y limitar red.

Rol	Finalidad	Recomendación
sagab_auditor	Consulta controlada de auditoría.	No usar como cuenta del backend normal.
sagab_readonly	BI/reportes de solo lectura.	Entregar solo a procesos autorizados.

No hay Row Level Security de PostgreSQL; el aislamiento contextual se implementa en servicios y consultas. Por ello las pruebas de autorización son un control crítico.

7. Seguridad y autorización

7.1 Autenticación

- API stateless, sin sesión HTTP de servidor.
- Contraseñas con BCrypt, factor de costo 12.
- JWT HMAC-SHA256 con duración de 30 minutos.
- Bloqueo después de 5 intentos fallidos durante 15 minutos.
- Cambio obligatorio de la clave temporal.
- Registro de acceso exitoso/fallido en eventos de seguridad.

`SecretsGuard` impide arrancar si no existen las variables `SAGAB_DB_PASSWORD` y `SAGAB_JWT_SECRET`, evitando usar silenciosamente los valores de ejemplo.

7.2 Superficie pública

Recurso	Motivo	Control compensatorio
<code>POST</code> <code>/api/auth/login</code>	Entrada al sistema.	Validación, bloqueo y auditoría.
<code>GET</code> <code>/api/storage/local/{token}</code>	Permite abrir/mostrar un archivo sin poder adjuntar Bearer desde ciertos contextos.	Token HMAC firmado, corto y emitido solo tras autorizar el recurso.

Todo otro endpoint requiere autenticación.

7.3 Autorización por capas

Rol de aplicación	Alcance	Restricción principal
<code>ADMIN</code>	Administración y operación institucional.	Acceso a matrícula, académico, asistencia, calendario, mensajes, finanzas y personal.
<code>DOCENTE</code>	Gestión académica de sus asignaciones.	Limitado a sus materias y paralelos.
<code>DECE</code>	Seguimiento y alertas estudiantiles.	Acceso especializado a información de acompañamiento.
<code>AUDITOR</code>	Consulta de trazabilidad y panel.	Sin operación académica.

Rol de aplicación	Alcance	Restricción principal
REPRESENTANTE	Consulta de representados.	Limitado a estudiantes vinculados.
ESTUDIANTE	Consulta y participación propia.	Limitado a su identidad estudiantil.

1. **SecurityConfig:** reglas generales por ruta/método.
2. **@PreAuthorize:** roles permitidos en controlador.
3. **Servicio:** propiedad y alcance: docente asignado, estudiante propio, representado vinculado, remitente autorizado.
4. **Base:** claves foráneas, unicidad, checks y triggers de integridad.

Alcance corregido. Docente ya no recibe promedio, rendimiento ni mora institucional; sus ausencias se limitan a paralelos activos asignados. `/dashboard/promedio` y `/finanzas/mora` quedan reservados a Administración y Auditoría según el endpoint.

7.4 Cabeceras, CORS y CSRF

- CORS acepta solo orígenes exactos de `SAGAB_CORS`, métodos GET/POST/PUT/PATCH/DELETE/OPTIONS y cabeceras Authorization/Content-Type.
- `X-Frame-Options: DENY` reduce clickjacking.
- `X-Content-Type-Options` está habilitado.
- HSTS se emite para un año e incluye subdominios; solo tiene efecto real sobre HTTPS.
- CSRF se desactiva porque la API usa Bearer y no cookie de autenticación.

7.5 Recomendaciones para producción

- HTTPS obligatorio, TLS moderno y redirección HTTP → HTTPS en proxy/plataforma.
- Secretos en un gestor, rotación, acceso mínimo y nunca en Git/logs.
- Base y bucket privados; firewall y credenciales distintas por entorno.
- Agregar Content-Security-Policy, Referrer-Policy y Permissions-Policy tras pruebas.
- Protección de tasa para login y endpoints costosos, además del bloqueo por cuenta.
- Antivirus/escaneo asíncrono de archivos si se aceptan documentos reales.
- Prueba OWASP, revisión de dependencias y modelo de amenazas antes del contrato.

8. Reglas de negocio relevantes

Módulo	Reglas actuales
Matrícula	Cédula ecuatoriana de 10 dígitos; edad normal 3–20 con advertencia para BGU; cuatro controles documentales; representante existente puede reutilizarse; se crean credenciales temporales.
Asignación	Una relación define docente, materia, paralelo y periodo. El docente puede acumular varias.
Notas	Escala 0–10; Tarea 20 %, Clase 20 %, Examen 60 %; aprobación ≥ 7 ; un registro por estudiante/asignación/parcial.
Asistencia	Fecha no futura; estados de dominio incluyen presente, ausencias y atraso; alerta desde 3 ausencias injustificadas consecutivas.
Deberes	Parcial 1–3; entrega/reemplazo antes del vencimiento; revisión docente con nota 1–10 opcional y retroalimentación; acceso por asignación/representado.
Calendario	Consulta máxima 370 días; no administradores ven publicados/cancelados y programados cuando llega la fecha; deberes pendientes solo para estudiante activo del paralelo; recursos por alcance.
Mensajes	Asunto ≤ 150 , cuerpo ≤ 10.000 ; destinatarios determinados por rol/asignación; registro individual de lectura.
Finanzas	Obligación por estudiante/rubro; pago aprobado no puede exceder saldo; transferencia requiere evidencia; hash SHA-256 detecta repetición para la obligación; vencimiento automático.
DECE	Estados ACTIVO, EN_OBSERVACION, INTERVENCION y RESUELTO; archivo lógico; historial append; observación ≤ 2.000 .
Auditoría	Cambios con before/after JSONB y eventos de seguridad; actualización y eliminación bloqueadas.

9. Catálogo de API

Prefijo base: `/api`. Salvo login y descarga local firmada, se requiere Bearer JWT. La tabla resume el contrato de rutas; los DTO Java son la fuente exacta de campos, tipos y validaciones.

9.1 Identidad y catálogos

Método y ruta	Rol	Función
POST <code>/auth/login</code>	Público	Autenticar y devolver sesión/JWT.
GET <code>/auth/me</code>	Autenticado	Reconstruir la sesión vigente.
POST <code>/auth/cambiar-clave</code>	Autenticado	Cambiar clave con validación de la actual.
GET <code>/paralelos</code>	ADMIN, DECE, AUDITOR	Catálogo de paralelos.
GET <code>/estudiantes/paralelo/{id}</code>	DOCENTE, ADMIN, DECE	Lista por paralelo con alcance de servicio.
GET <code>/estudiantes/mios</code>	REPRESENTANTE, ESTUDIANTE	Representados o identidad estudiantil propia.
GET <code>/estudiantes/buscar</code>	ADMIN, DOCENTE	Búsqueda limitada.
GET <code>/materias/estudiante/{id}</code>	ADMIN, REPRESENTANTE, ESTUDIANTE	Materias visibles para el estudiante.

9.2 Personal, matrícula y asignaciones

Método y ruta	Rol	Función
POST <code>/personal</code>	ADMIN	Crear cuentas Docente, DECE o Auditor.
GET <code>/personal</code>	ADMIN	Listar personal operativo.
POST <code>/matriculas</code>	ADMIN	Matricular estudiante y vincular familia/cuentas.
GET <code>/asignaciones/mias</code>	DOCENTE, ADMIN	Asignaciones propias o todas para Administración.
GET <code>/asignaciones/catalogos</code>	ADMIN	Docentes, materias, paralelos y periodos para formulario.
POST <code>/asignaciones</code>	ADMIN	Crear asignación.
PUT <code>/asignaciones/{id}</code>	ADMIN	Editar asignación.

Método y ruta	Rol	Función
DELETE /asignaciones/{id}	ADMIN	Eliminar asignación.

9.3 Calificaciones y asistencia

Método y ruta	Rol	Función
POST /calificaciones	DOCENTE, ADMIN	Guardar lote/registro de notas.
GET /calificaciones/asignacion/{id}/parcial/{p}	DOCENTE, ADMIN	Planilla por asignación/parcial.
GET /calificaciones/estudiante/{id}	DOCENTE, ADMIN, REP., EST.	Notas del estudiante bajo control de propiedad.
GET /calificaciones/buscar	DOCENTE, ADMIN	Búsqueda avanzada.
GET /calificaciones/{idEstudiante}/papeleta	DOCENTE, ADMIN	Generar papeleta PDF.
DELETE /calificaciones/{id}	DOCENTE, ADMIN	Eliminar si el alcance lo permite.
POST /asistencia	DOCENTE, ADMIN	Guardar asistencia.
GET /asistencia/paralelo/{id}	DOCENTE, ADMIN, DECE	Consultar por paralelo/fecha.
GET /asistencia/paralelo/{id}/consecutivas	DOCENTE, ADMIN, DECE	Ausencias injustificadas consecutivas.
GET /asistencia/estudiante/{id}	DOCENTE, ADMIN, REP., EST.	Historial bajo alcance.
GET /asistencia/reporte	ADMIN, DOCENTE, AUDITOR, DECE	Reporte institucional; el docente queda limitado a sus paralelos.

9.4 Deberes y recursos

Método y ruta	Rol	Función
POST /tareass, PUT /tareass/{id}, DELETE /tareass/{id}	DOCENTE	Ciclo de vida de tareas propias.
GET /tareass/asignacion/{id}	DOCENTE, ADMIN	Lista por asignación.

Método y ruta	Rol	Función
GET /tareas/{id}/entregas	DOCENTE, ADMIN	Entregas de una tarea.
GET /tareas/estudiante/{id}	DOCENTE, ADMIN, REP., EST.	Deberes por estudiante con ownership.
POST /tareas/{idTarea}/estudiante/{id}/entrega	REPRESENTANTE, ESTUDIANTE	Subir/reemplazar entrega multipart.
POST /tareas/entregas/{id}/revisar	DOCENTE	Nota y retroalimentación.
GET /tareas/entregas/{id}/descarga	DOCENTE, ADMIN, REP., EST.	Descarga autorizada.
GET/POST /tareas/{id}/adjuntos	Lectura autorizada / DOCENTE	Listar o adjuntar material.
GET/DELETE /tareas/adjuntos/{id}	Lectura autorizada / DOCENTE	Descargar o eliminar adjunto.
GET /recursos-academicos/asignacion/{id}	ADMIN, DOCENTE, REP., EST.	Listar recursos visibles.
POST /recursos-academicos/archivo	DOCENTE	Crear recurso de archivo.
POST /recursos-academicos/link	DOCENTE	Crear recurso de enlace.
PATCH/DELETE /recursos-academicos/{id}	DOCENTE	Editar metadatos o eliminar recurso propio.
GET /recursos-academicos/{id}/descarga	ADMIN, DOCENTE, REP., EST.	Descarga bajo alcance.

9.5 Calendario

Método y ruta	Rol	Función
GET /calendario	Todos los roles	Eventos, feriados y capas visibles por rango/rol.
POST /calendario	ADMIN	Crear evento.
PUT /calendario/{id}	ADMIN	Editar evento.
POST /calendario/{id}/duplicar	ADMIN	Duplicar evento.
DELETE /calendario/{id}	ADMIN	Eliminar evento.
POST /calendario/{id}/adjuntos	ADMIN	Adjuntar multipart.

Método y ruta	Rol	Función
GET /calendario/adjuntos/{id}/descarga	Todos los roles	Descarga después de verificar visibilidad.
DELETE /calendario/adjuntos/{id}	ADMIN	Eliminar adjunto.

9.6 Mensajes, notificaciones y panel

Método y ruta	Rol	Función
GET /mensajes/mias	Autenticado	Bandeja recibida.
POST /mensajes	ADMIN	Mensaje directo administrativo.
GET /mensajes/enviados	Autenticado	Historial emitido bajo remitente.
POST /mensajes/broadcast	ADMIN, DOCENTE	Envío grupal con restricciones.
POST /mensajes/institucionales	ADMIN	Envío masivo/privado a docentes.
POST /mensajes/{id}/leido	Autenticado	Marcar lectura propia.
GET /notificaciones/mias	Autenticado	Notificaciones del usuario.
POST /notificaciones/{id}/leida	Autenticado	Marcar notificación propia.
GET /dashboard/resumen	ADMIN, DOCENTE, AUDITOR	Indicadores; requiere corrección de scope.
GET /dashboard/promedio	ADMIN, DOCENTE, AUDITOR	Detalle/promedio; requiere corrección de scope.

9.7 Finanzas

Método y ruta	Rol	Función
GET /finanzas/estudiante/{id}	ADMIN, REP., EST.	Obligaciones y pagos propios/autorizados.
GET /finanzas/mora	ADMIN, DOCENTE, AUDITOR, DECE	Listado institucional actual; alcance a endurecer.
POST /finanzas/mora/{id}/comunicacion	ADMIN, DOCENTE	Recordatorio/comunicación actual; revisar necesidad docente.

Método y ruta	Rol	Función
POST /finanzas/pagos	ADMIN	Registrar pago directo.
POST /finanzas/obligaciones	ADMIN	Crear obligación.
POST /finanzas/pagos/transfere ncia	REP., EST.	Reportar transferencia con comprobante.
GET /finanzas/rubros	ADMIN, REP., EST.	Catálogo de rubros visibles.
GET /finanzas/pagos/revisio n	ADMIN	Cola de transferencias.
POST /finanzas/pagos/{id}/aproba r	ADMIN	Aprobar tras validar.
POST /finanzas/pagos/{id}/rechaz ar	ADMIN	Rechazar con motivo.
GET /finanzas/pagos/{id}/compro bante	ADMIN, REP., EST.	Abrir evidencia bajo ownership.

9.8 DECE y auditoría

Método y ruta	Rol	Función
GET /dece/seguimientos/estudiantes	DECE	Buscar estudiantes activos.
GET/POST /dece/seguimientos	DECE	Listar o crear seguimiento.
GET/PUT/DELETE /dece/seguimientos/{id}	DECE	Detalle, actualización y archivo lógico.
GET /dece/seguimientos/{id}/historial	DECE	Línea de tiempo.
POST/GET /dece/seguimientos/{id}/mensajes	DECE	Enviar y consultar comunicaciones del caso.
GET /auditoria/cambios	AUDITOR, ADMIN	Consulta paginada de cambios.
GET /auditoria/eventos	AUDITOR, ADMIN	Eventos de seguridad.
GET /auditoria/historial/{tabla}/{idFila}	AUDITOR, ADMIN	Trazabilidad de una fila.

Método y ruta	Rol	Función
GET /storage/local/{token}	Token firmado	Descarga local temporal sin Bearer.

10. Almacenamiento y archivos

10.1 Selección de proveedor

Si están presentes todas las variables esenciales S3 —endpoint, bucket, access key y secret key— se usa el proveedor compatible. Si todas faltan, se usa almacenamiento local. Una configuración parcial debe rechazarse para evitar escribir archivos en un destino inesperado.

Modo	Uso	Riesgo/condición
Local	Desarrollo o servidor con volumen persistente.	Un filesystem efímero pierde adjuntos al desplegar; requiere volumen y respaldo.
S3 compatible	Producción distribuida: AWS S3, R2 u otro compatible.	Bucket privado, CORS del bucket, política mínima, cifrado y lifecycle.

10.2 Controles

- Claves/nombres internos aleatorios y defensa contra recorrido de rutas.
- Comprobación de extensión, MIME declarado y firmas/magic bytes.
- Autorización antes de emitir descarga.
- En modo local, URL HMAC firmada con caducidad aproximada de 10 minutos.
- Hash SHA-256 del comprobante financiero para detectar duplicado en una obligación.

10.3 Límites

Tipo	Límite lógico	Configuración
Entrega/deber y calendario	15 MB por archivo	SAGAB_UPLOAD_MAX_MB_DEBERES
Comprobante	8 MB	SAGAB_UPLOAD_MAX_MB_COMPROBANTE
Recurso académico	100 MB	SAGAB_UPLOAD_MAX_MB_RECURSOS
Multipart global	100 MB por archivo/petición por defecto	SAGAB_UPLOAD_MAX_MB
Adjuntos de evento	10 archivos	Regla de servicio

El límite global debe ser igual o mayor al máximo lógico. El proxy/CDN también debe aceptar el tamaño necesario; si su límite es menor, responderá antes que Spring.

11. Configuración y variables

11.1 Backend

Variable	Obligatoria	Descripción / ejemplo no secreto
<code>SAGAB_DB_HOST</code>	Producción	Host PostgreSQL; local por defecto.
<code>SAGAB_DB_PORT</code>	No	5432 por defecto.
<code>SAGAB_DB_NAME</code>	Producción	<code>sagab</code> por defecto.
<code>SAGAB_DB_USER</code>	Producción	Usuario de aplicación, normalmente <code>sagab_app</code> .
<code>SAGAB_DB_PASSWORD</code>	Sí	Secreto de base; SecretsGuard exige presencia.
<code>SAGAB_JWT_SECRET</code>	Sí	Al menos 32 bytes aleatorios; generar con <code>openssl rand -base64 48</code> .
<code>SAGAB_CORS</code>	Producción	Origen exacto del frontend; admite lista según binding Spring.
<code>PORT</code>	No	Puerto HTTP; 8080 por defecto.
<code>SAGAB_PUBLIC_API_URL</code>	Producción	URL pública HTTPS usada al formar enlaces.
<code>SAGAB_STORAGE_LOCAL_DIR</code>	Modo local	Directorio persistente; default <code>./data/uploads</code> .
<code>SAGAB_S3_ENDPOINT</code>	Modo S3	Endpoint del proveedor.
<code>SAGAB_S3_REGION</code>	Modo S3	Región; default <code>auto</code> .
<code>SAGAB_S3_BUCKET</code>	Modo S3	Bucket privado.
<code>SAGAB_S3_ACCESS_KEY</code>	Modo S3	Clave de acceso mínima.
<code>SAGAB_S3_SECRET_KEY</code>	Modo S3	Secreto; nunca registrar.
<code>SAGAB_SMTP_HOST</code>	No	Servidor SMTP.
<code>SAGAB_SMTP_USER</code>	Si correo	Usuario SMTP.
<code>SAGAB_SMTP_PASS</code>	Si correo	Secreto SMTP.
<code>SAGAB_UPLOAD_MAX_MB*</code>	No	Ajustes de límites descritos en capítulo 10.

11.2 Frontend

Variable	Momento	Descripción
VITE_API_URL	Build	URL pública del backend, sin barra final; ej. <code>https://api.colegio.edu.ec</code> .

Vite incorpora variables `VITE_*` en el bundle: nunca colocar secretos allí. El repositorio no contiene actualmente un `.env.example` operativo aunque algunos textos antiguos lo sugieran; debe crearse uno sin valores sensibles como mejora de mantenimiento.

12. Instalación local

12.1 Prerrequisitos

- Git.
- Java 17 y Maven 3.9.
- Node.js 20 LTS y npm.
- PostgreSQL 15+ con `psql` y `createdb` en PATH.
- Puertos 5173 y 8080 disponibles.

12.2 Preparar PostgreSQL

```
cd /ruta/al/proyecto/sagab
bash database/setup_db.sh
```

El script supone que el usuario del sistema puede listar y crear bases con las credenciales locales configuradas en PostgreSQL. En un servidor administrado, aplique los scripts con una cuenta de migración y no con la cuenta runtime.

12.3 Iniciar backend

```
cd backend
export SAGAB_DB_HOST=localhost
export SAGAB_DB_PORT=5432
export SAGAB_DB_NAME=sagab
export SAGAB_DB_USER=sagab_app
export SAGAB_DB_PASSWORD='valor-local-seguro'
export SAGAB_JWT_SECRET="$(openssl rand -base64 48)"
export SAGAB_CORS=http://localhost:5173
mvn spring-boot:run
```

Verifique el arranque en el log y que no aparezcan fallos de validación Hibernate. No copie el secreto generado a documentación, tickets o Git.

12.4 Iniciar frontend

```
cd frontend
npm ci
# Crear manualmente .env con esta línea no secreta:
# VITE_API_URL=http://localhost:8080
npm run dev
```

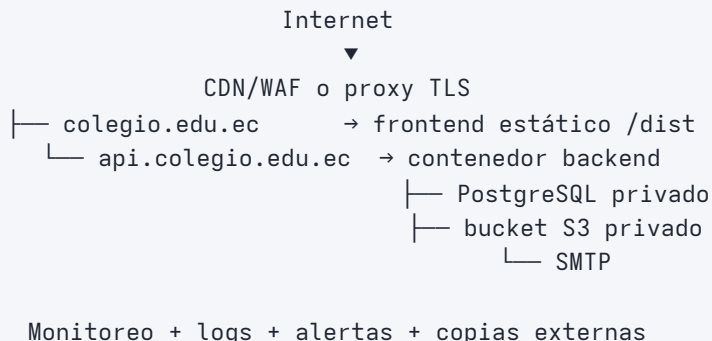
Abra `http://localhost:5173`. Para exponer temporalmente en la red local, use `npm run dev -- --host 0.0.0.0`, ajuste CORS al origen exacto y permita el puerto en firewall. No es una configuración de producción.

12.5 Verificación mínima

1. Login correcto e incorrecto.
2. `/api/auth/me` conserva sesión al recargar.
3. Cada rol ve su menú y recibe 403 en operaciones ajenas.
4. Consulta de cursos/paralelos y asignaciones.
5. Creación/consulta en un módulo no destructivo de prueba.
6. Subida y descarga de un archivo de prueba permitido.
7. Cerrar sesión elimina el token.

13. Despliegue de producción

13.1 Topología recomendada



13.2 Backend con Docker

El `backend/Dockerfile` usa dos etapas: Maven 3.9/Temurin 17 compila el JAR y Temurin 17 JRE Alpine lo ejecuta. El build actual omite pruebas con `-DskipTests`; la canalización debe ejecutar `mvn test` antes de construir la imagen.

```
docker build -t sagab-backend:1.0 ./backend
docker run --rm -p 8080:8080 \
  --env-file /ruta-segura/sagab-backend.env \
  sagab-backend:1.0
```

El archivo de entorno debe vivir fuera del repositorio y con permisos restringidos. Si se usa almacenamiento local, monte un volumen explícito; preferentemente use S3 compatible.

13.3 Frontend estático

```
cd frontend
npm ci
VITE_API_URL=https://api.colegio.edu.ec npm run build
```

Publique `frontend/dist` en un host estático y configure fallback de rutas hacia `index.html`. La URL API queda embebida en build; un cambio de entorno requiere reconstrucción.

13.4 Base y migraciones

1. Crear base, roles y secretos rotados.
2. Respalidar antes de cada cambio.

3. Aplicar scripts estructurales en el orden de `setup_db.sh` con una cuenta de migración.
4. Revisar salida y versión; ejecutar una prueba de arranque con `ddl-auto=validate`.
5. Desplegar aplicación y ejecutar smoke tests.

13.5 Observación sobre render.yaml

La plantilla actual aprovisiona PostgreSQL y backend en plan gratuito, pero no incluye el frontend y mantiene un placeholder en `SAGAB_CORS`. Tampoco aplica automáticamente los 24 scripts estructurales. Los planes gratuitos, el disco efímero y la ausencia de SLA no son apropiados para información institucional sin una evaluación y controles adicionales.

14. Operación, respaldo y recuperación

14.1 Responsabilidades diarias

Frecuencia	Control
Continuo	Disponibilidad HTTP, errores 5xx, latencia, uso de conexiones, almacenamiento y fallos de login anómalos.
Diario	Estado de backup, jobs de vencimiento, capacidad, errores de storage/SMTP y cola financiera.
Semanal	Revisión de seguridad, cuentas activas, crecimiento de auditoría y restauración de muestra.
Mensual	Partición de auditoría, parches, dependencias, capacidad y revisión de accesos.
Por periodo	Catálogos, asignaciones, políticas de notas, archivo y retención.

14.2 Política mínima de respaldo

- PostgreSQL: backup diario automático y copias incrementales/PITR si el proveedor lo permite.
- Archivos: versionado o réplica del bucket/volumen.
- Secretos/configuración: inventario cifrado y recuperable por dos responsables.
- Copias fuera del mismo proveedor o zona de fallo.
- Retención definida por calendario académico, normativa y contrato.
- Prueba de restauración completa al menos trimestral y antes de cambios mayores.

Ejemplo conceptual; sustituir host, rol y ruta mediante variables seguras y no colocar la contraseña en la línea de comandos:

```
pg_dump -Fc --no-owner --no-acl -h DB_HOST -U BACKUP_ROLE sagab \  
-f /backup/sagab_YYYYMMDD.dump  
  
createdb sagab_restore_test  
pg_restore --clean --if-exists --no-owner -d sagab_restore_test \  
/backup/sagab_YYYYMMDD.dump
```

14.3 Objetivos de recuperación

Rectorado y proveedor deben acordar formalmente RPO (pérdida máxima de datos aceptable) y RTO (tiempo máximo de recuperación). Valores orientativos como RPO 24 h/RTO 8 h no son una garantía del código; dependen del plan contratado y deben probarse.

14.4 Procedimiento de incidente

1. Registrar hora, síntoma, alcance, despliegue reciente y quién reporta.
2. Proteger evidencia y limitar accesos; no modificar auditoría.
3. Clasificar disponibilidad, confidencialidad, integridad o operación.
4. Contener: revocar secretos/sesiones, aislar servicio o desactivar función según riesgo.
5. Recuperar desde versión/backup verificado y validar procesos críticos.
6. Notificar según contrato y normativa; documentar causa y acciones preventivas.

15. Pruebas y calidad

15.1 Comandos de control

```
cd backend
mvn test

cd ../frontend
npm ci
npm run type-check
npm run build
```

15.2 Cobertura existente

El repositorio contiene pruebas unitarias para fechas de Ecuador, validación de archivos, papeleta PDF y almacenamiento. Estas pruebas validan piezas específicas, no un sistema completo.

15.3 Matriz requerida antes de producción

Tipo	Cobertura mínima pendiente	Prioridad
Integración API/DB	Transacciones, restricciones, scripts desde cero, concurrencia de pagos y auditoría.	Crítica
Autorización	Cada endpoint por rol, propiedad y tentativa de IDOR; especialmente dashboard, mora, reportes y archivos.	Crítica
End-to-end	Login, matrícula, asignación, notas, asistencia, tarea, entrega, calendario, transferencia, DECE y auditoría.	Crítica
Seguridad	OWASP, cargas maliciosas, JWT, CORS, enumeración, fuerza bruta, errores y dependencias.	Crítica
Carga	Usuarios concurrentes objetivo, dashboard, búsquedas, reportes y carga/descarga.	Alta
Recuperación	Restaurar DB + archivos y verificar consistencia cruzada.	Crítica
Compatibilidad	Chrome, Edge, Firefox; escritorio, tableta, teléfono.	Alta
Accesibilidad	Teclado, foco, lector de pantalla, contraste y formularios; WCAG 2.1 AA.	Alta

No se detectó una canalización CI en el repositorio. Debe impedir merge/despliegue si fallan pruebas, type-check, build, análisis de dependencias o migración de prueba.

16. Diagnóstico y errores

Síntoma	Causa probable	Diagnóstico
No arranca: falta variable.	<code>SecretsGuard</code> .	Definir <code>SAGAB_DB_PASSWORD</code> y <code>SAGAB_JWT_SECRET</code> en el proceso real.
Hibernate validation failed.	Script faltante o modelo/esquema distintos.	Comparar migraciones aplicadas; no cambiar a <code>ddl-auto=update</code> como atajo.
Connection refused/timeout DB.	Host, puerto, firewall, SSL o credenciales.	Probar <code>psql</code> desde el mismo entorno y revisar pool.
CORS en navegador.	Origen no coincide exactamente.	Comparar esquema, host y puerto con <code>SAGAB_CORS</code> ; reiniciar backend.
401 al recargar.	Token expirado/secret cambiado/reloj.	Revisar expiración, hora y estabilidad de <code>SAGAB_JWT_SECRET</code> .
403 en operación legítima.	Rol, asignación o ownership inconsistente.	Revisar identidad, relaciones y logs; no ampliar roles globalmente sin análisis.
Archivos desaparecen.	Disco efímero o ruta no montada.	Configurar bucket/volumen y restaurar copia.
413 al subir.	Límite proxy o Spring menor.	Alinear proxy, multipart global y límite lógico.
Falla S3 al arrancar/usar.	Variables parciales, endpoint, reloj o permisos.	Validar conjunto completo, política y conectividad; no hacer bucket público.
No llega correo.	SMTP vacío/rechazado.	Revisar STARTTLS, cuenta, secreto y logs; el mensaje interno es independiente.
Error creando evento.	Rango inválido, estado/fecha, archivo o esquema no migrado.	Confirmar script 28, fechas inicio≤fin y límites.
Cursos vacíos.	Periodo/paralelo/asignación/matricula faltante.	Verificar catálogos y relaciones con cuenta autorizada.

Logs. Mantener INFO en producción, agregar un identificador de correlación y centralizar logs. No habilitar cuerpos HTTP, SQL con parámetros o DEBUG global en un entorno con datos reales.

17. Capacidades y límites técnicos

Elemento	Límite/estado	Efecto
Pool DB	Máx. 20, mínimo idle 5.	Ajustar con carga, conexiones disponibles e instancias.
Sesión	JWT 30 minutos, sin refresh activo.	Reautenticación frecuente; revocación inmediata limitada por stateless.
Calendario	Rango máximo 370 días; refresco UI 30 s.	No consultar años ilimitados en una llamada.
Resultados	Varias búsquedas hasta 500; revisión 200; mensajes/notificaciones 100; auditoría 50/página.	No equivale a exportación completa ni paginación uniforme.
Archivos	8/15/100 MB y 10 adjuntos por evento.	Proxy y storage deben alinearse.
Frontend	SPA con vistas diferidas; formularios y fuentes optimizados; chunk de gráficos pesado.	Medir build de producción en perfil limpio y redes móviles.
Escala	Sin benchmark de concurrencia.	No prometer cantidad de usuarios ni SLA sin prueba.
API	Sin OpenAPI/Swagger versionado.	Integración externa más costosa y riesgo de deriva documental.
Observabilidad	Logs INFO; sin Actuator/métricas/alertas integrales.	Detección y diagnóstico limitados.
Migraciones	Scripts manuales sin tabla formal de versión.	Mayor riesgo de orden, repetición y diferencias entre ambientes.

18. Riesgos, deuda técnica y hoja de ruta

18.1 Bloqueantes antes de producción comercial

1. **Validar integralmente autorización y privacidad.** La matriz y el alcance del panel fueron corregidos; falta probar acceso negativo de todos los endpoints y roles en integración/E2E.
2. **Crear pruebas de integración y E2E.** Cubrir flujos críticos y cada endpoint por rol/ownership.
3. **Automatizar CI/CD seguro.** Pruebas, type-check, build, escaneo, imagen, migración y smoke test.
4. **Definir backup/restore y almacenamiento persistente.** Ejecutar una restauración comprobable de DB y adjuntos.
5. **Desplegar HTTPS con monitoreo.** Métricas, alertas, logs, incidentes y SLA contratados.
6. **Gestión administrativa de cuentas.** Desactivar, editar, restablecer y revocar sesiones con auditoría.
7. **Evaluación de privacidad y seguridad.** Datos de menores, financieros y DECE exigen mínimo privilegio, retención y respuesta a incidentes.

18.2 Prioridad alta

- Migrar scripts a Flyway/Liquibase y eliminar credenciales/cuentas personales de scripts de distribución.
- Implementar cierre de parciales, autorización de correcciones y trazabilidad académica amigable.
- Completar matrícula: lista, edición, retiro/promoción/rematrícula y expedientes.
- Completar asistencia: atraso, justificativo, evidencia y aprobación.
- Crear catálogos administrativos de periodo, materia, paralelo y rubro.
- Documentar API mediante OpenAPI y añadir versionado.
- Mejorar auditoría visual, filtros y exportación autorizada.

18.3 Evolución funcional

- Facturación electrónica e integración contable solo después de análisis regulatorio.
- Planificación, horarios, rúbricas y actas académicas.
- Flujos especializados de DECE con compartimentación y consentimiento.
- Eventos recurrentes e integración con calendarios externos.
- Canales de notificación adicionales con consentimiento y trazabilidad.

19. Listas de verificación

19.1 Antes de cada despliegue

- ☐ Cambio revisado y vinculado a requisito/incidente.
- ☐ `mvn test`, `npm run type-check` y `npm run build` exitosos.
- ☐ Pruebas de autorización y regresión del módulo.
- ☐ Dependencias y secretos revisados.
- ☐ Backup confirmado y rollback definido.
- ☐ Migración ensayada en copia y base limpia.
- ☐ CORS, API URL, storage, SMTP y zona correctos.
- ☐ Ventana, responsable y comunicación aprobados.

19.2 Después del despliegue

- ☐ Salud del frontend, API, DB y storage.
- ☐ Login y recuperación de sesión.
- ☐ Un caso de lectura por cada rol.
- ☐ Operación crítica controlada y descarga/subida.
- ☐ Jobs y partición de auditoría.
- ☐ Sin aumento anómalo de 401, 403 o 5xx.
- ☐ Versión y resultado documentados.

19.3 Criterio de salida a producción

La salida se autoriza cuando Rectorado, responsable técnico y responsable de datos aceptan el alcance; los bloqueantes del capítulo 18 están cerrados o formalmente tratados; existe evidencia de pruebas, restauración y seguridad; y la institución dispone de soporte, privacidad, continuidad y capacitación.

Responsable técnico
Nombre, firma y fecha

Responsable institucional
Nombre, firma y fecha